

exercise2-recsys-2024

March 15, 2024

1 Content-based Recommenders

1.1 Overview

During this exercise you will create and use some content-based profiles for generating predictions using Python. It will be necessary to make different computational variations to observe how certain features of computations can bring biases.

Please upload in Moodle this notebook (executable and in PDF) with the code and answers to the posted questions.

1.2 Instructions

Before you start just to remind the intuitive algorithm of the content-based filtering:

1. Computing vectors to describe items (have a dataset of described articles (items)).
2. Based on the user ratings of items and item's content profiles, build a user profile preferences (preferences of attributes).
3. Predicting user interests in items based on a dot product of the item profile vector and user profile vector. Vectors should be normalized.
4. From this prediction create a top n recommendations or scale for rating-scale predictions

```
[3]: from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
[12]: #AUTHOR: Ariton Verush

import numpy as np
import pandas as pd

# read CSV
movies = pd.read_csv('/content/movies.csv')

#Users ratings. 1 means like, -1 dislike, 0 not rated
```

```
john_likes=pd.DataFrame([1, -1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0,
↪-1, 0])
joan_likes=pd.DataFrame([-1, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, -1, 0, 0, 0, 0, 1,
↪0, 0, 0])

#Adding the likes to the Dataset
movies["John_Likes"]=john_likes
movies["Joan_Likes"]=joan_likes

#The dataframe with the dataset
print(movies)

#Copy of the original dataset useful for Task 2
movies_weighted=movies.copy()
```

	Movie	Adventure	Animation	Children	\
0	Toy Story (1995)	1	0	1	
1	Jumanji (1995)	0	1	1	
2	Grumpier Old Men (1995)	0	0	0	
3	Waiting to Exhale (1995)	0	0	1	
4	Father of the Bride Part II (1995)	0	1	0	
5	Heat (1995)	1	0	0	
6	Sabrina (1995)	0	0	0	
7	Tom and Huck (1995)	0	0	1	
8	Sudden Death (1995)	0	0	0	
9	GoldenEye (1995)	0	1	0	
10	American President, The (1995)	0	0	1	
11	Dracula: Dead and Loving It (1995)	1	0	0	
12	Balto (1995)	0	0	1	
13	Nixon (1995)	0	1	1	
14	Cutthroat Island (1995)	0	0	0	
15	Casino (1995)	1	0	0	
16	Sense and Sensibility (1995)	0	1	1	
17	Four Rooms (1995)	0	0	0	
18	Ace Ventura: When Nature Calls (1995)	0	1	1	
19	Money Train (1995)	0	0	1	

	Comedy	Fantasy	Romance	Drama	Thriller	Action	Horror	Num_Attr	\
0	0	1	1	0	0	0	1	5	
1	1	0	0	0	1	0	0	4	
2	1	1	1	0	0	0	0	3	
3	1	0	0	1	1	0	0	4	
4	0	0	0	0	0	1	1	3	
5	1	0	0	0	0	0	0	2	
6	0	0	0	0	1	0	1	2	
7	1	0	0	1	0	0	1	4	
8	0	0	1	0	0	1	0	2	

9	0	1	0	1	0	0	0	3
10	0	1	0	0	0	1	0	3
11	0	0	1	1	0	0	0	3
12	1	1	0	0	1	0	0	4
13	1	0	0	0	0	1	0	4
14	1	0	1	1	1	0	0	4
15	0	0	1	0	0	1	0	3
16	1	0	0	0	1	0	0	4
17	1	0	0	0	0	1	0	2
18	0	1	0	1	0	0	1	5
19	1	0	0	1	0	1	0	4

	John_Likes	Joan_Likes
0	1	-1
1	-1	1
2	0	0
3	0	1
4	0	0
5	1	0
6	0	0
7	0	0
8	0	0
9	0	0
10	0	0
11	0	-1
12	0	0
13	0	0
14	0	0
15	1	0
16	0	1
17	0	0
18	-1	0
19	0	0

1.2.1 Preparation

For this exercise, the small dataset `movies.csv` will be used. It has twenty (20) entries and each entry corresponds to the name of a movie, followed by ten values which can be one (1) or zero (0), where one (1) means that the movie belongs to the genre indicated on the header of the csv file; the last value represents the number of genres that the movie has (`Num_Attr`) 1. Open the file `CBRecommender.py` with your favorite Python editor. 2. Read the code following the instructions of this document. 3. Complete the tasks by adding the lines of code that you consider necessary.

Task 1: Build a recommender with simple unweighted content-based profiles

Build up a user profile of preferences of movie genre for the user *Joan*. Write the necessary lines of codes, to this end you may follow these instructions and example:

- Count the total number of positive and negative evaluations associated with each movie genre,

and then create a profile with the total score for each genre for user *Joan*.

- For example, *John's* score for the genre “Adventure” will be equal to three (3) from three (3) positive evaluations of the movies Toy Story, Heat, and casino.
- Basically, you need to use a dot-product-like operation of the user's ratings dataframe (i.e., `john_likes`) with the attribute's vector across the twenty (20) movies. Take as an example of this the following line of code:

```
[13]: #User profile for John by movie genre
john_likes_score=pd.DataFrame((movies.iloc[:,1:11].values*john_likes.values).
    ↪sum(axis=0)).T
print(john_likes_score)
```

```

    0  1  2  3  4  5  6  7  8  9
0  3 -2 -1  0  0  2 -1 -1  1  0
```

Compute the user profile for Joan:

You could use something similar as the code above to perform the calculations for user *Joan*.

```
[14]: #User profile for Joan by movie genre
joan_likes_score=pd.DataFrame((movies.iloc[:,1:11].values*joan_likes.values).
    ↪sum(axis=0)).T
print(joan_likes_score)
```

```

    0  1  2  3  4  5  6  7  8  9
0 -2  2  2  3 -1 -2  0  3  0 -1
```

Now, you have the user profile of preferences for each movie genre and also the movie vector with content attributes (the genres of each movie). From this, compute the predicted score for each article for the user *Joan* as explained below and answer the following questions: 1. Produce a dot-product of user's profile preferences with each movie vector.

Example for John:

```
[15]: #Prediction vales for user John
pred_john=(movies.iloc[:,1:11].values*john_likes_score.values).sum(axis=1)

movies["Pred_John"]=pred_john

#Showing prediction scores for John and the movies provided
print(movies[['Movie', 'John_Likes', 'Pred_John']])
```

	Movie	John_Likes	Pred_John
0	Toy Story (1995)	1	4
1	Jumanji (1995)	-1	-4
2	Grumpier Old Men (1995)	0	2
3	Waiting to Exhale (1995)	0	-3
4	Father of the Bride Part II (1995)	0	-1
5	Heat (1995)	1	3
6	Sabrina (1995)	0	-1
7	Tom and Huck (1995)	0	-2

8	Sudden Death (1995)	0	3
9	GoldenEye (1995)	0	-3
10	American President, The (1995)	0	0
11	Dracula: Dead and Loving It (1995)	0	4
12	Balto (1995)	0	-2
13	Nixon (1995)	0	-2
14	Cutthroat Island (1995)	0	0
15	Casino (1995)	1	6
16	Sense and Sensibility (1995)	0	-4
17	Four Rooms (1995)	0	1
18	Ace Ventura: When Nature Calls (1995)	-1	-4
19	Money Train (1995)	0	-1

```
[17]: # Write here the code to calculate the prediction vales for user Joan

pred_joan=(movies.iloc[:,1:11].values*joan_likes_score.values).sum(axis=1)

movies["Pred_Joan"]=pred_joan

# Showing prediction scores for Joan and the movies provided
print(movies[['Movie', 'Joan_Likes', 'Pred_Joan']])
```

	Movie	Joan_Likes	Pred_Joan
0	Toy Story (1995)	-1	-4
1	Jumanji (1995)	1	10
2	Grumpier Old Men (1995)	0	0
3	Waiting to Exhale (1995)	1	8
4	Father of the Bride Part II (1995)	0	1
5	Heat (1995)	0	1
6	Sabrina (1995)	0	2
7	Tom and Huck (1995)	0	4
8	Sudden Death (1995)	0	-2
9	GoldenEye (1995)	0	1
10	American President, The (1995)	0	1
11	Dracula: Dead and Loving It (1995)	-1	-4
12	Balto (1995)	0	7
13	Nixon (1995)	0	7
14	Cutthroat Island (1995)	0	4
15	Casino (1995)	0	-4
16	Sense and Sensibility (1995)	1	10
17	Four Rooms (1995)	0	3
18	Ace Ventura: When Nature Calls (1995)	0	2
19	Money Train (1995)	0	5

2. Which movie does the simple profile predict *John* will like the most, excluding the movies he has already rated? > Answer: “Dracula: Dead and Loving It”
3. What score does that prediction get? > Answer: 4

4. How many movies does the model predict the user *Joan* will dislike i.e., have a negative score?
> Answer: 2 (excluding the movies that Joan already dislikes)
5. Which movie will the user *Joan* like the most, excluding the movies she has already rated?
> Answer: Both of the movies “Balto” and “Nixon” have the same prediction score of 7, and they haven’t been rated by Joan so far (neither 1 or -1)

Task 2: Build a recommender with simple weighted contentbased profiles

It can be noticed that, in the previous computation a movie that has many genres checked could have more influence on the overall user profile than one that had only a few. For instance, the movie Toy Story has five (5) genres, while Sabrina only has two (2).

In this task you will explore whether the previous model (Task 1) may be counting attribute-heavy documents too much resulting in biased prediction scores. For this, write the code that you consider necessary, simply add it under the indicated comment. You may use the dataframe `movies_weighted`, which is a copy of the original dataframe but without the profiles and prediction scores of the previous task (check the first code block).

1. Build a new weighted model by doing the following:
 - Get the square root of the total number of attributes for each movie. You can use the field `Num_Attr` of the dataset for this.
 - Divide each value of each article vector by the value calculated in the previous point

```
[20]: # Write your code here
# Calculate the square root of the total attributes for each movie

import numpy as np

Num_Attr=np.sqrt(movies.iloc[:,11])
pred_john=(movies.iloc[:,1:11].values*john_likes_score.values).sum(axis=1)

movies_weighted["Pred_John"]=pred_john/Num_Attr

#Showing prediction scores for John and the movies provided
print(movies_weighted[['Movie','John_Likes','Pred_John']])
```

	Movie	John_Likes	Pred_John
0	Toy Story (1995)	1	1.788854
1	Jumanji (1995)	-1	-2.000000
2	Grumpier Old Men (1995)	0	1.154701
3	Waiting to Exhale (1995)	0	-1.500000
4	Father of the Bride Part II (1995)	0	-0.577350
5	Heat (1995)	1	2.121320
6	Sabrina (1995)	0	-0.707107
7	Tom and Huck (1995)	0	-1.000000
8	Sudden Death (1995)	0	2.121320
9	GoldenEye (1995)	0	-1.732051
10	American President, The (1995)	0	0.000000
11	Dracula: Dead and Loving It (1995)	0	2.309401

12	Balto (1995)	0	-1.000000
13	Nixon (1995)	0	-1.000000
14	Cutthroat Island (1995)	0	0.000000
15	Casino (1995)	1	3.464102
16	Sense and Sensibility (1995)	0	-2.000000
17	Four Rooms (1995)	0	0.707107
18	Ace Ventura: When Nature Calls (1995)	-1	-1.788854
19	Money Train (1995)	0	-0.500000

2. Compute your new user profiles and new predictions, using the new attribute values and the same schema as in Task 1.

[21]: *# Write your code here*

```
import numpy as np

Num_Attr=np.sqrt(movies.iloc[:,11])
pred_joan=(movies.iloc[:,1:11].values*joan_likes_score.values).sum(axis=1)

movies_weighted["Pred_Joan"]=pred_joan/Num_Attr

#Showing prediction scores for Joan and the movies provided
print(movies_weighted[['Movie','Joan_Likes','Pred_Joan']])
```

	Movie	Joan_Likes	Pred_Joan
0	Toy Story (1995)	-1	-1.788854
1	Jumanji (1995)	1	5.000000
2	Grumpier Old Men (1995)	0	0.000000
3	Waiting to Exhale (1995)	1	4.000000
4	Father of the Bride Part II (1995)	0	0.577350
5	Heat (1995)	0	0.707107
6	Sabrina (1995)	0	1.414214
7	Tom and Huck (1995)	0	2.000000
8	Sudden Death (1995)	0	-1.414214
9	GoldenEye (1995)	0	0.577350
10	American President, The (1995)	0	0.577350
11	Dracula: Dead and Loving It (1995)	-1	-2.309401
12	Balto (1995)	0	3.500000
13	Nixon (1995)	0	3.500000
14	Cutthroat Island (1995)	0	2.000000
15	Casino (1995)	0	-2.309401
16	Sense and Sensibility (1995)	1	5.000000
17	Four Rooms (1995)	0	2.121320
18	Ace Ventura: When Nature Calls (1995)	0	0.894427
19	Money Train (1995)	0	2.500000

3. Have a look at the *John's* and *Johan's* prediction scores. Answer the following questions:"

- Which article does the new model predict John will like the best, excluding the movies he

has already rated? Is it different than in Task 1? > Answer: “Dracula: Dead and Loving it”. The answer is not different, because it is the same movie from Task 1. The only difference is that the prediction scores are more precise (in decimals).

- What score does that prediction get? > Answer: 2.309401
- How many movies does the new model predict the user Joan will dislike? > Answer: 2, excluding the movies that Joan already dislikes (she doesn’t like “Toy Story” and “Dracula: Dead and Loving it”)
- Which movie will the user Joan like the most, excluding the movies she has already rated? Any changes in comparison to the results of Task 1? > Answer: The answer is the same, the movies are again “Balto” and “Nixon”. The change is that we get more precise predictions(in decimals), so both now have a prediction score of 3.500000.
- Which model do you consider is better? Under which situation would you use the unweighted model and under which the weighted model? > Answer: It is better to use a weighted model, since it provides us with more precise prediction scores. However, when we have equally important values and attributes, using the unweighted model can satisfy the needs. The choice between a weighted and unweighted model depends on the specific requirements of the recommendation task, the characteristics of the dataset, and other considerations take part as well. It may also be a good approach to experiment with both models and evaluate their performance, and after that we can determine which model better meets the needs of our application.